IL1332 Digital Design using HDL

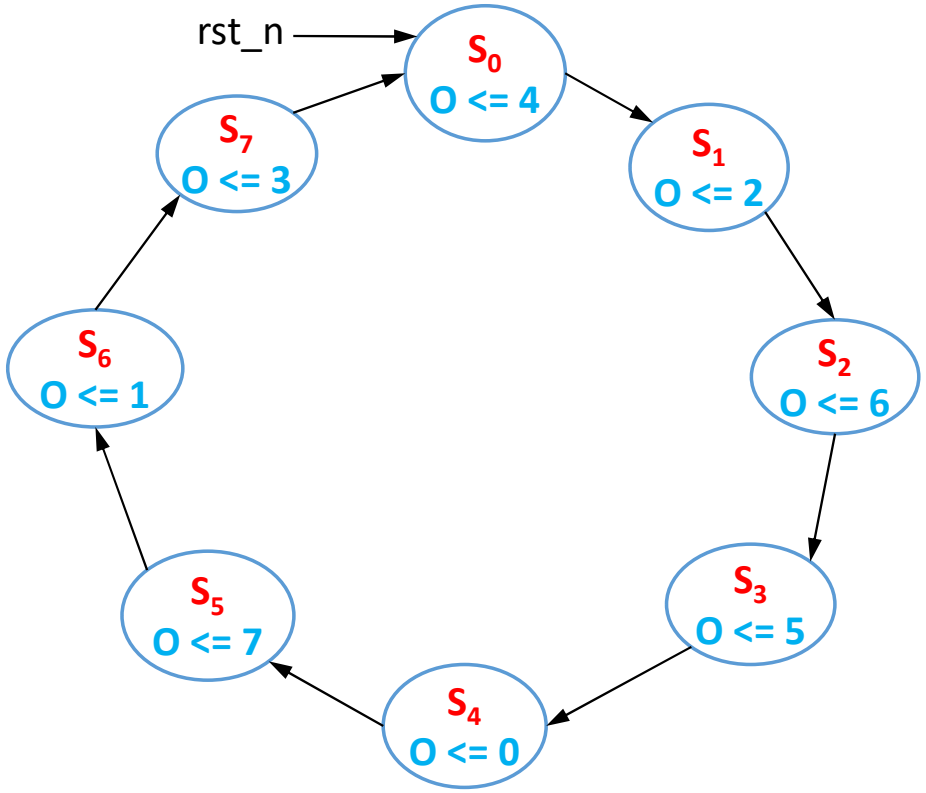
L15: Synthesis of Counters & Traffic Light FSM

Ahmed Hemani

School of EECS, KTH, Stockholm, Sweden

Synthesis of Counters as FSMs

Synthesis of arbitrary sequence counter using D Flops



Sequence (notice the arbitrary order)

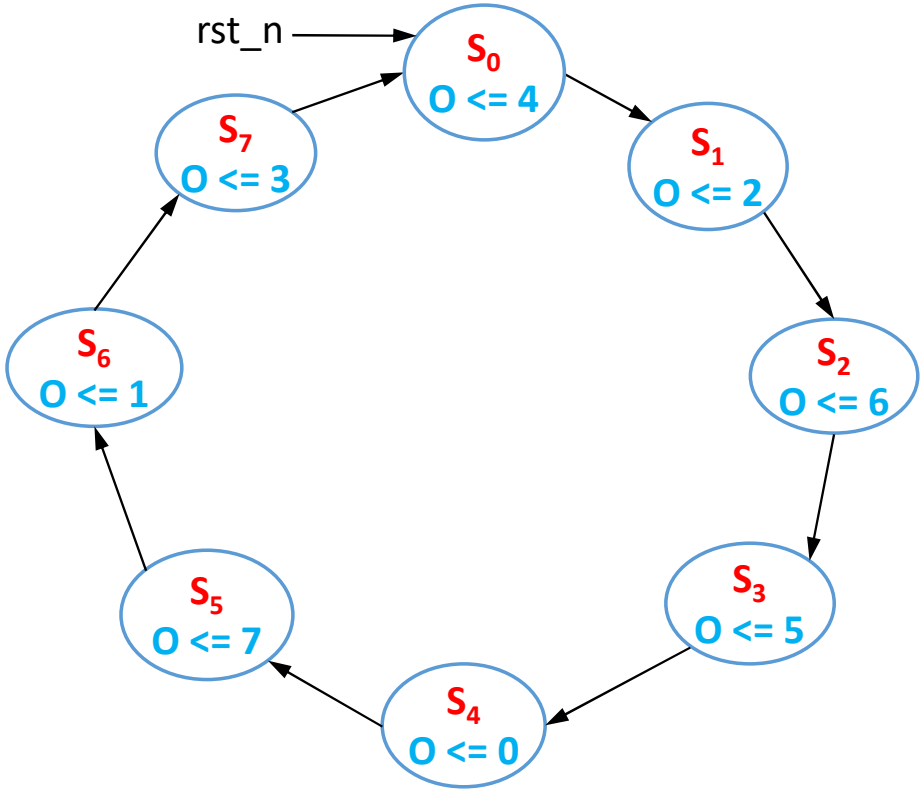
```
4 - 100
2 - 010
6 - 110
5 - 101
0 - 000
7 - 111
1 - 001
3 - 011
```

You could use the same method to synthesize Gray Code Sequence Counter

© Ahmed Hemani, KTH, Stockholm

Arbitrary Counter Sequence Example

Counters can also be modelled as an FSM



1. *There is only one transition in and one transition out*
2. *The transitions are unconditional*
3. *It is naturally modelled as a Moore FSM*
4. *There are 8 states → 3 Flops*
5. *The output of Flops will directly encode the output. Count = f(state-code); f is an identity function, i.e. Count = State-Code*

   The states will be encoded as follows:

   a. *S_0 will be encoded as 100*
   b. *S_1 will be encoded as 010*
   c. *S_2 will be encoded as 110*
   d. *And so on*

© Ahmed Hemani, KTH, Stockholm

```
module counter_arb_seq( input  logic clk,
                        input  logic rst_n,
                        output logic [2:0] count);

typedef enum  logic[2:0] {S_0 = 3'b100,
                          S_1 = 3'b010,
                          S_2 = 3'b110,
                          S_3 = 3'b101,
                          S_4 = 3'b000,
                          S_5 = 3'b111,
                          S_6 = 3'b001,
                          S_7 = 3'b011 } state_ty;

   state_ty pres_state, next_state;

always_ff @(posedge clk or negedge rst_n) begin
      if (!rst_n) begin
            pres_state <= S_0;
      end
      else begin
            pres_state <= next_state;
      end
   end
```

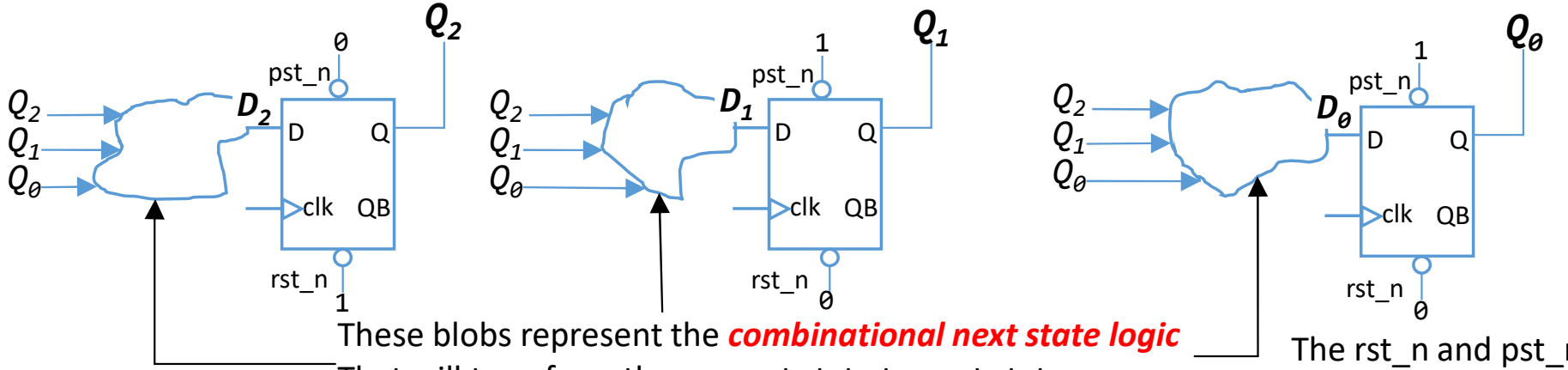
```
always_comb begin
    case (pres_state)
      S_0: next_state = S_1;
      S_1: next_state = S_2;
      S_2: next_state = S_3;
      S_3: next_state = S_4;
      S_4: next_state = S_5;
      S_5: next_state = S_6;
      S_6: next_state = S_7;
      S_7: next_state = S_0;
    endcase
   end // always_comb begin

   assign count = pres_state;

endmodule
```


© Ahmed Hemani, KTH, Stockholm

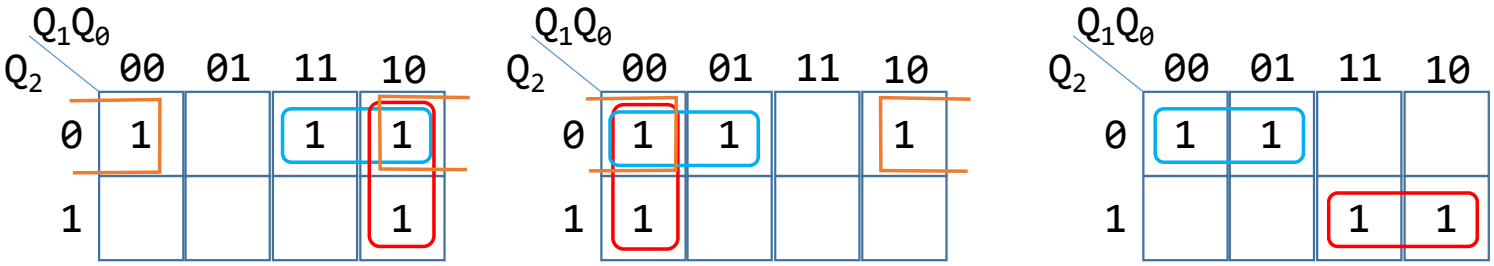
The Structure of the Counter



These blobs represent the ***combinational next state logic***
That will transform the present state to next state

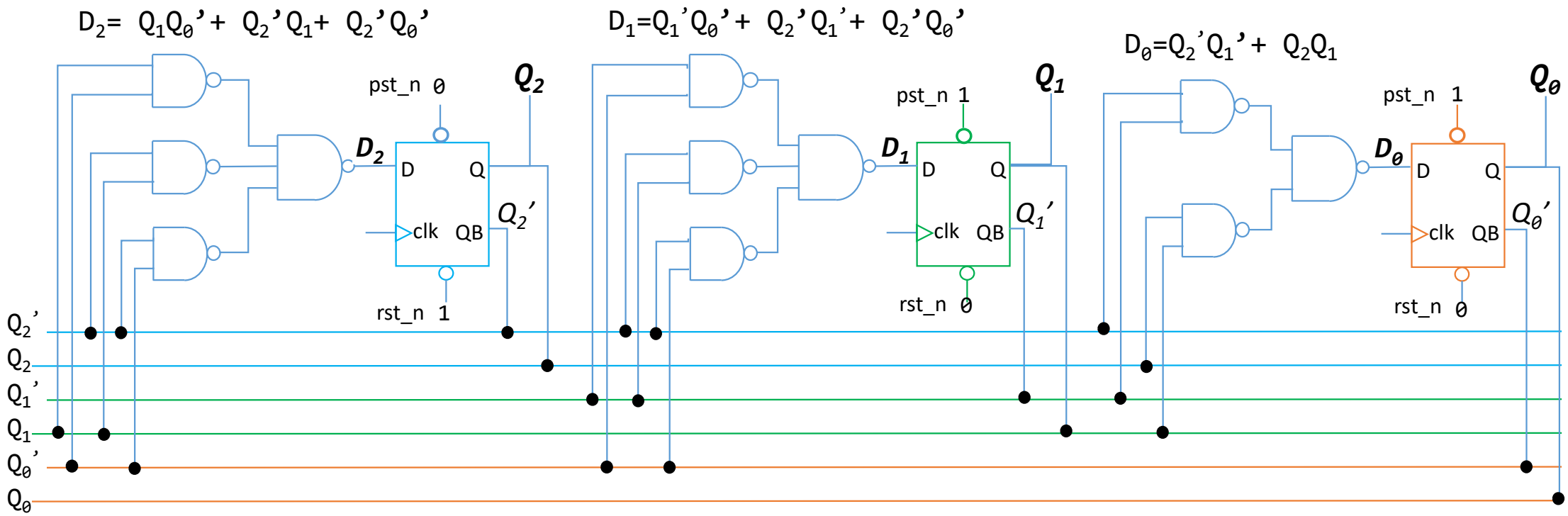
The rst_n and pst_n are used to start the counter in state 100

Present State			Next State		
Q_2	Q_1	Q_0	D_2	D_1	D_0
1	0	0	0	1	0
0	1	0	1	1	0
1	1	0	1	0	1
1	0	1	0	0	0
0	0	0	1	1	1
1	1	1	0	0	1
0	0	1	0	1	1
0	1	1	1	0	0



© Ahmed Hemani, KTH, Stockholm

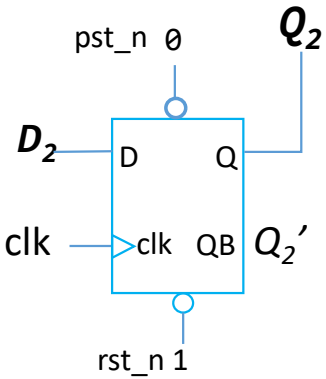
The Arbitrary Sequence Counter Using D Flops



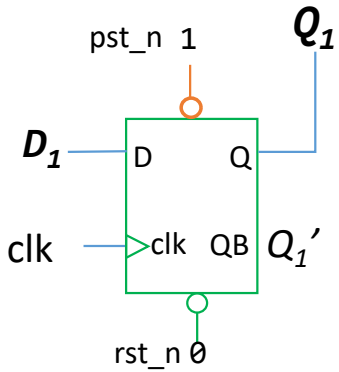
© Ahmed Hemani, KTH, Stockholm

In Exams and recitations – this will be enough

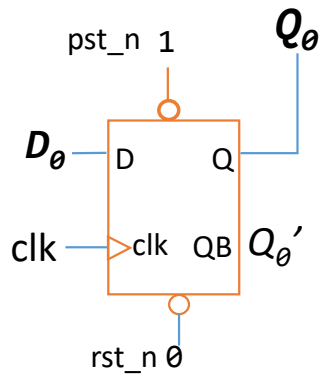
$D_2 = Q_1Q_0' + Q_2'Q_1 + Q_2'Q_0'$



$D_1 = Q_1'Q_0' + Q_2'Q_1' + Q_2'Q_0'$



$D_0 = Q_2'Q_1' + Q_2Q_1$



© Ahmed Hemani, KTH, Stockholm

```
module counter_arb_seq_impl( input  logic clk,
                             input  logic rst_n,
                             output logic [2:0] count);

   parameter N = 1;
   parameter time Setup = 1ns;
   parameter time Clk2Q = 1.5ns;

   logic [2:0]          Q, QB, D;
   logic     rst_n2, rst_n1, rst_n0;
   logic     pre_n2, pre_n1, pre_n0;

   d_reg #(N, Setup, Clk2Q) R2 (.D(D[2]),
                                .clk(clk),
                                .rst_n(rst_n2),
                                .pre_n(pre_n2),
                                .Q(Q[2]),
                                .QB(QB[2]));

   d_reg #(N, Setup, Clk2Q) R1 ( .D(D[1]),
                                 .clk(clk),
                                 .rst_n(rst_n1),
                                 .pre_n(pre_n1),
                                 .Q(Q[1]),
                                 .QB(QB[1]));

   d_reg #(N, Setup, Clk2Q) R0 ( .D(D[0]),
                                 .clk(clk),
                                 .rst_n(rst_n0),
                                 .pre_n(pre_n0),
                                 .Q(Q[0]),
                                 .QB(QB[0]));
```

```
always_comb begin
      D[2] = Q[1] & QB[0] | QB[2]& Q[1] | QB[2] & QB[0];
      D[1] = QB[1] & QB[0] | QB[2]& QB[1] | QB[2] & QB[0];
      D[0] = QB[2] & QB[1] | Q[2]& Q[1];

   end // always_comb begin

   always_comb begin
      if (rst_n == 0) begin
            rst_n2 = 1;
            pre_n2 = 0;

            rst_n1 = 0;
            pre_n1 = 1;

            rst_n0 = 0;
            pre_n0 = 1;
      end // if (rst_n == 0)

      else begin
            rst_n2 = 1;
            pre_n2 = 1;

            rst_n1 = 1;
            pre_n1 = 1;

            rst_n0 = 1;
            pre_n0 = 1;
      end // else: !if(rst_n == 0)

   end // always_comb begin

   assign count = Q;

endmodule
```

© Ahmed Hemani, KTH, Stockholm

```
`timescale 1ns/1ps

module tb_counter_arb_seq;

  logic clk;
  logic rst_n;
  logic [2:0] count, count_impl;

  counter_arb_seq uut1 (
    .clk(clk),
    .rst_n(rst_n),
    .count(count));

  counter_arb_seq_impl uut2 (
    .clk(clk),
    .rst_n(rst_n),
    .count(count_impl));

  always #10 clk = ~clk;

  initial begin
    clk = 0;
    rst_n = 0;

    // Apply reset
    rst_n = 0;

    #25; // Hold reset for 25 ns
    rst_n = 1;

    #200;
    $stop;
  end
endmodule
```

© Ahmed Hemani, KTH, Stockholm

Q12. 5p

Question:

Design a 4-bit synchronous counter that cycles through the odd prime numbers.

When the control input signal UP = 1, the sequence should be 3, 5, 7, 11, 13, 3 . . . etc.

When the control input signal UP = 0, the sequence should be 3, 13, 11, 7, 5, 3 . . . etc.

The signal UP can change in any state but is synchronous with the clock.

Design the counter using D flip-flops. (A variation could be to design using JK, T or SR Flops)

Hint: since all allowed numbers are odd, $q_0 = 1$, and only q_3, q_2 and q_1 need logic and flip-flops

a) (1p) Make a state table for the allowed states.

b) (2p) Using Karnaugh Maps design the logic expressions for next state q_3+, q_2+ and q_1+.

c) (1p) Draw a state diagram with all states, including the forbidden 1, 9, and 15.

Why are 2,4,6,8,10,12, and 14 not forbidden states ?

d) (1p) Design asynchronous set/reset logic so that the counter can always be set in state 3 at start

© Ahmed Hemani, KTH, Stockholm

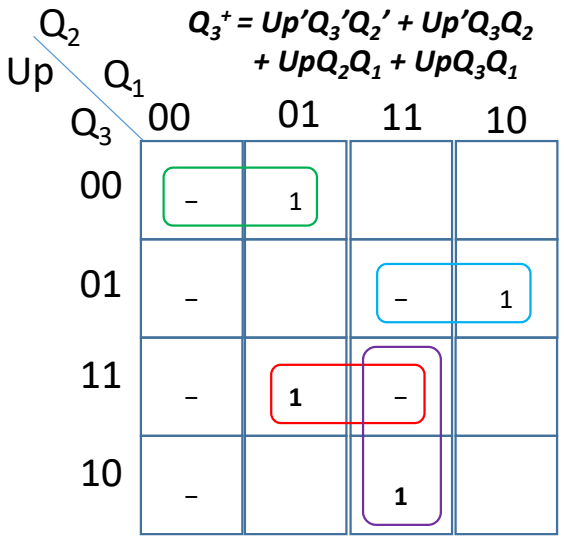
Notice that Q_0 and Q_0^+ is permanently "1" because we are dealing with prime numbers from 3 to 13.
For this reason, we do not need next state logic or a flop for the LSB Q_0. We show them in table for completeness

Input	Present State				Output / Next State			
Up	Q_3	Q_2	Q_1	Q_0	Q_3^+	Q_2^+	Q_1^+	Q_0^+
1	0	0	1	1	0	1	0	1
1	0	1	0	1	0	1	1	1
1	0	1	1	1	1	0	1	1
1	1	0	1	1	1	1	0	1
1	1	1	0	1	0	0	1	1
0	0	0	1	1	1	1	0	1
0	0	1	0	1	0	0	1	1
0	0	1	1	1	0	1	0	1
0	1	0	1	1	0	1	1	1
0	1	1	0	1	1	0	1	1

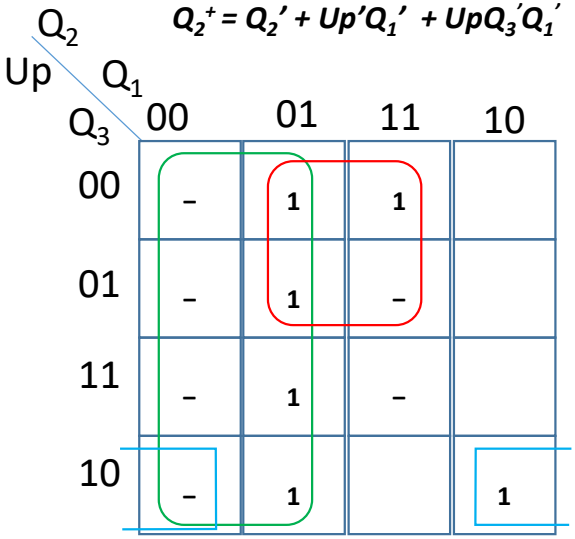
Don't Care Minterms

Up	Q_3	Q_2	Q_1	Q_0
0	0	0	0	1
0	1	0	0	1
0	1	1	1	1
1	0	0	0	1
1	1	0	0	1
1	1	1	1	1

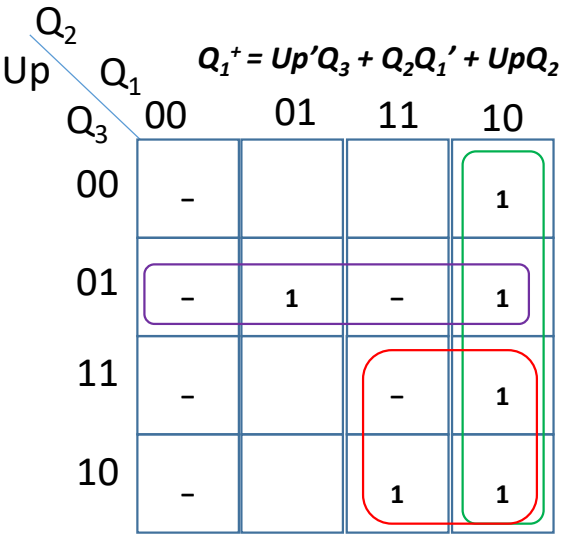
$Q_3^+ = Up'Q_3'Q_2' + Up'Q_3Q_2 + UpQ_2Q_1 + UpQ_3Q_1$



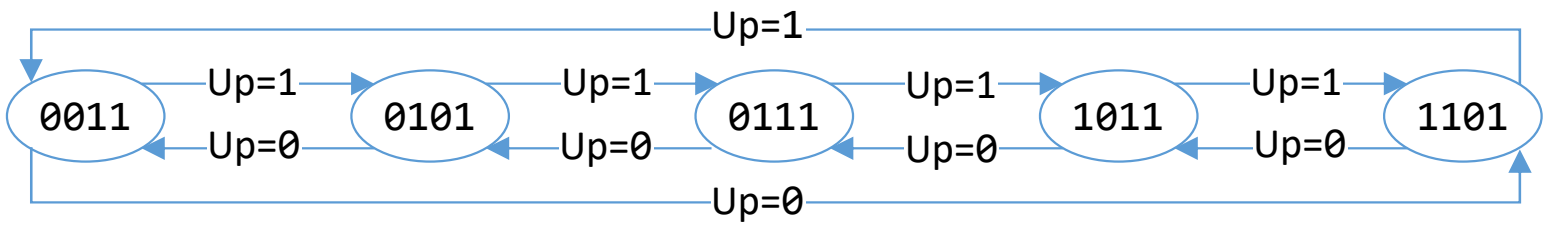
$Q_2^+ = Q_2' + Up'Q_1' + UpQ_3'Q_1'$



$Q_1^+ = Up'Q_3 + Q_2Q_1' + UpQ_2$



© Ahmed Hemani, KTH, Stockholm



What are the forbidden or illegal states ?

1. Since the LSB Q_0 is permanently set to 1 because all states are odd numbers
2. We ***can only have odd states***. There are 8 odd states
3. Of these eight odd states,
4. Five are legal – the prime numbers from 3 to 13
5. And the other three – *0001, 1001, 1111* – are illegal

How do we find out what are the next states, should the State machine accidentally land in one of these illegal states?

1. We plug in the illegal present state Q_3, Q_2 and Q_1 into the three next state equations to find out the next state. Alternatively, we can consult the K-Map to find out the next states for these three illegal present states

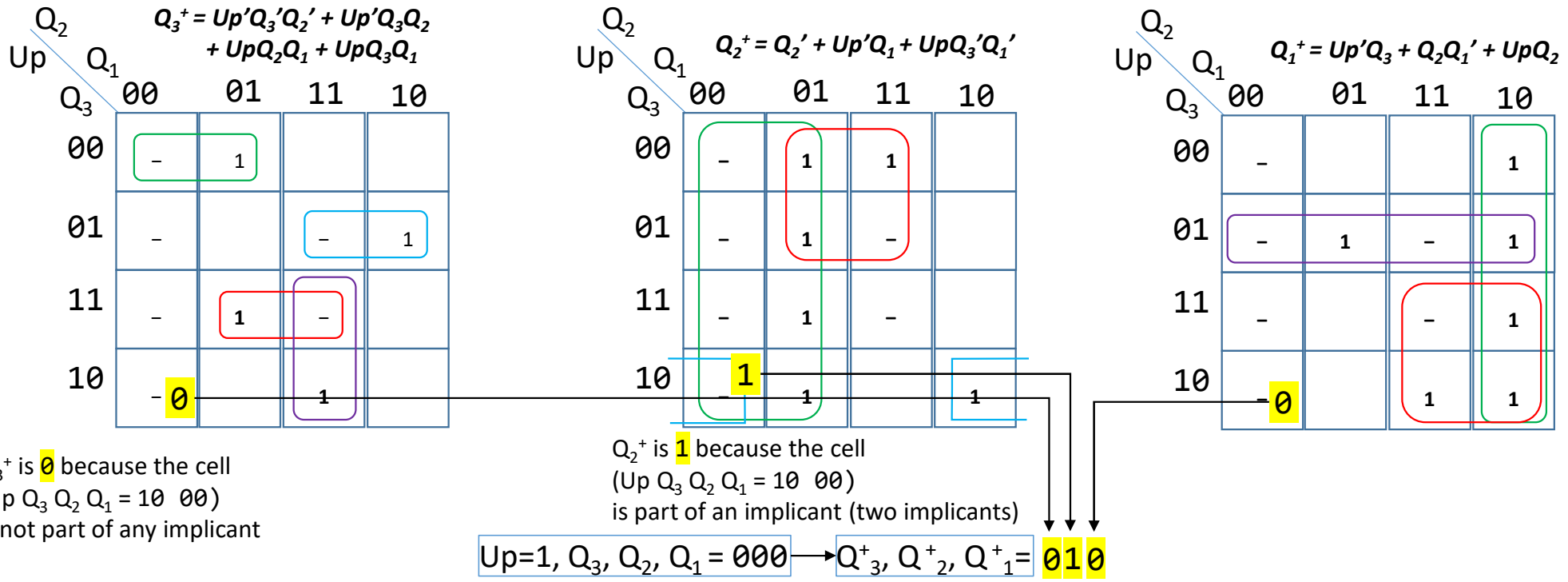
$Q_3^+ = Up'Q_3'Q_2' + Up'Q_3Q_2 + UpQ_2Q_1 + UpQ_3Q_1$ $Q_2^+ = Q_2' + Up'Q_1' + UpQ_3'Q_1'$ $Q_1^+ = Up'Q_3 + Q_2Q_1' + UpQ_2$

1. Up=1, Q_3, Q_2, Q_1, Q_0 = 0001, Q_3^+, Q_2^+, Q_1^+, Q_0^+ = 0101 and Up=0, Q_3, Q_2, Q_1, Q_0 = 0001, Q_3^+, Q_2^+, Q_1^+, Q_0^+= 1101
2. Up=1, Q_3, Q_2, Q_1, Q_0 = 1001, Q_3^+, Q_2^+, Q_1^+, Q_0^+ = 0101 and Up=0, Q_3, Q_2, Q_1, Q_0 = 1001, Q_3^+, Q_2^+, Q_1^+, Q_0^+ = 0111
3. Up=1, Q_3, Q_2, Q_1, Q_0 = 1111, Q_3^+, Q_2^+, Q_1^+, Q_0^+ = 1011 and Up=0, Q_3, Q_2, Q_1, Q_0 = 1111, Q_3^+, Q_2^+, Q_1^+, Q_0^+ = 1111

© Ahmed Hemani, KTH, Stockholm

Reading Next State from Karnaugh Map

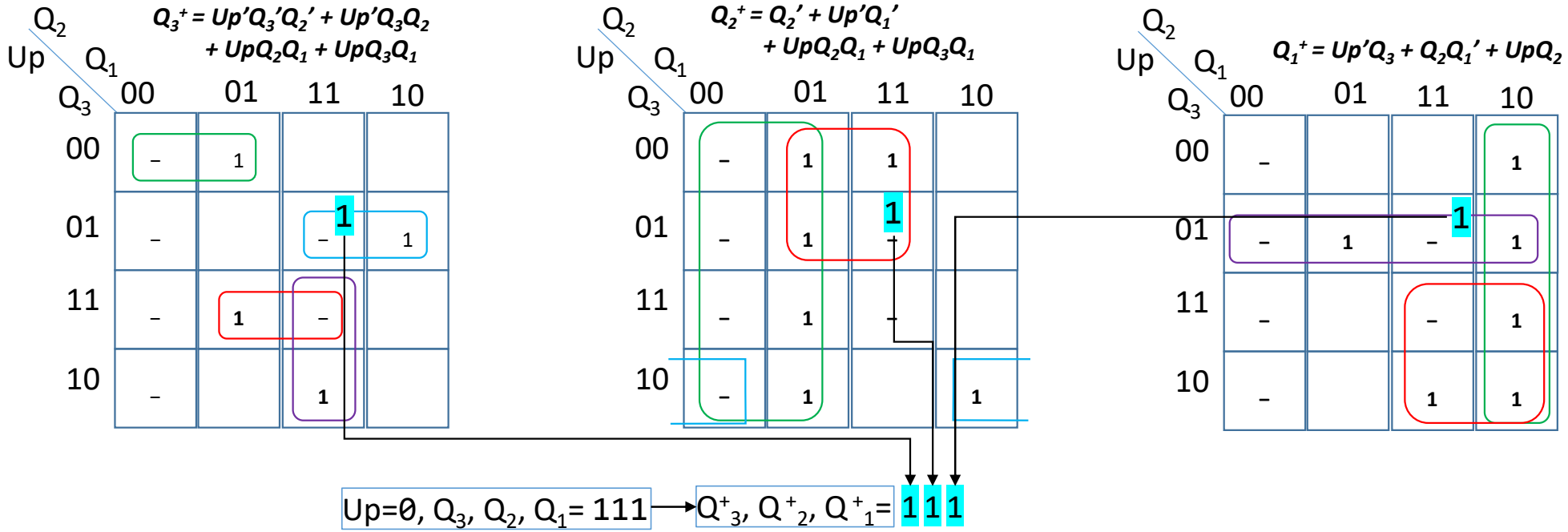
1. *Illegal Sates:* $Q_3\ Q_2\ Q_1\ Q_0$ = 0001, 1001, 1111 – illegal States.
   Q_0 is ignored because it is tied to 1
2. The next states for these illegal states can be for *Up = 1 or Up = 0*



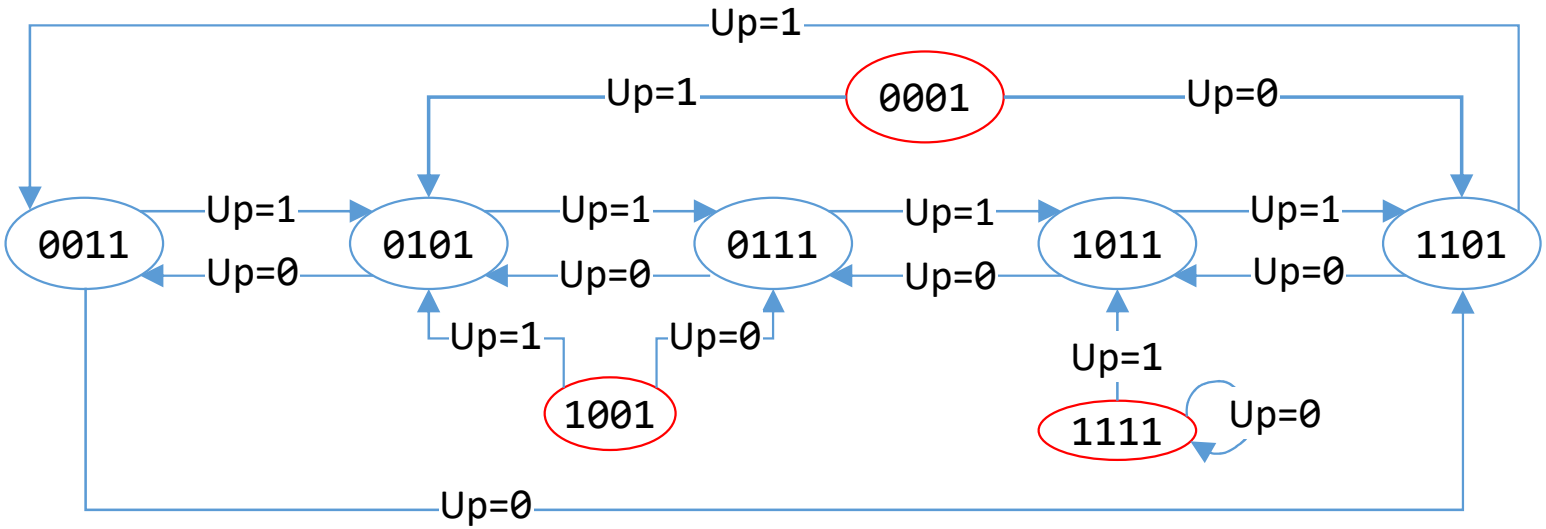
© Ahmed Hemani, KTH, Stockholm

Reading Next State from Karnaugh Map

1. *Illegal Sates:* $Q_3\ Q_2\ Q_1\ Q_0$ = *0001, 1001, 1111* – illegal States.
   Q_0 is ignored because it is tied to 1
2. The next states for these illegal states can be for *Up = 1 or Up = 0*



© Ahmed Hemani, KTH, Stockholm

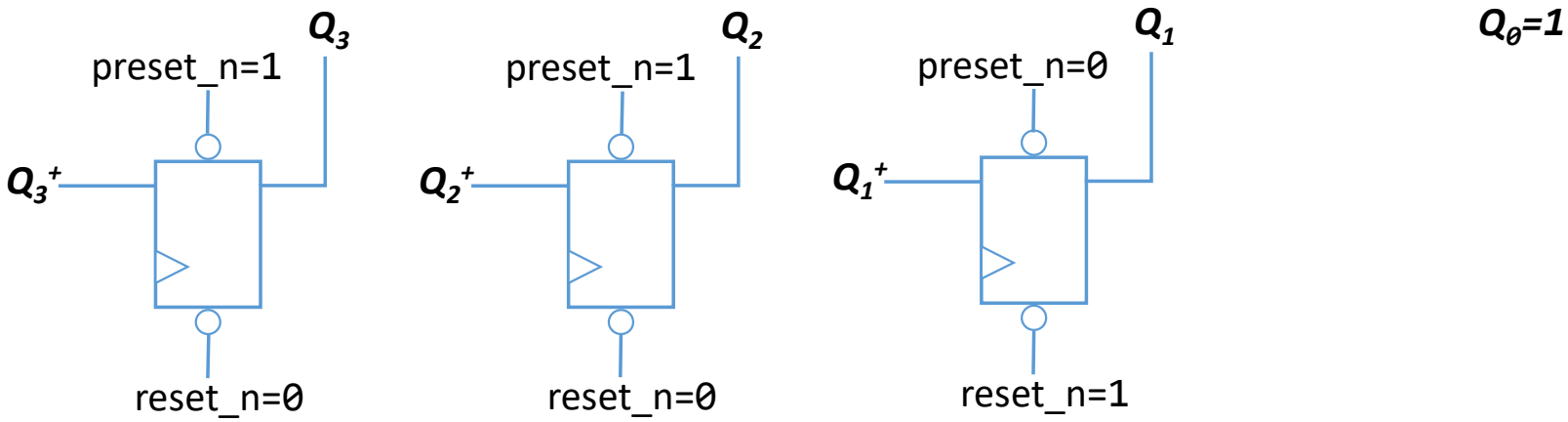


Remember that Q_0 and Q_0^+ is permanently tied to logic 1
For this reason Q_3,Q_2,Q_1= 000 means present state Q_3,Q_2,Q_1,Q_0 = 0001
And for the same reason Q_3^+,Q_2^+,Q_1^+= 010 means next state Q_3^+,Q_2^+,Q_1^+,Q_0^+= = 0101

1. For Up=1,Q_3,Q_2,Q_1,Q_0 = 0001, Q_3^+,Q_2^+,Q_1^+,Q_0^+= 0101 and Up=0,Q_3,Q_2,Q_1,Q_0 = 0001, Q_3^+,Q_2^+,Q_1^+,Q_0^+= 1101
2. For Up=1,Q_3,Q_2,Q_1,Q_0 = 1001, Q_3^+,Q_2^+,Q_1^+,Q_0^+= 0101 and Up=0,Q_3,Q_2,Q_1,Q_0 = 1001, Q_3^+,Q_2^+,Q_1^+,Q_0^+= 0111
3. For Up=1,Q_3,Q_2,Q_1,Q_0 = 1111, Q_3^+,Q_2^+,Q_1^+,Q_0^+= 1011 and Up=0,Q_3,Q_2,Q_1,Q_0 = 1111, Q_3^+,Q_2^+,Q_1^+,Q_0^+= 1111

© Ahmed Hemani, KTH, Stockholm

Design asynchronous set/reset logic so that the counter will always be in state 3 at start



$Q_3^+ = Up'Q_3'Q_2' + Up'Q_3Q_2 + UpQ_2Q_1 + UpQ_3Q_1$

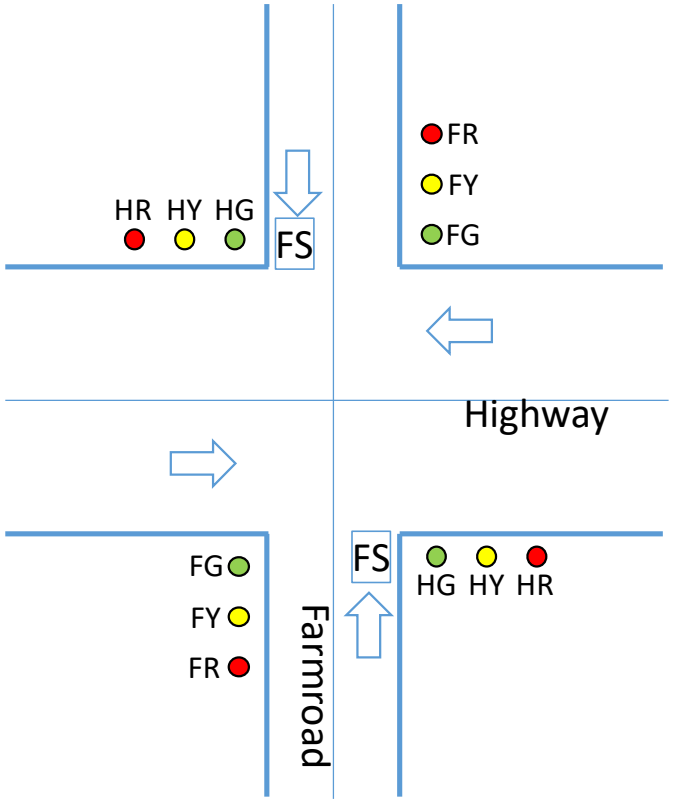
$Q_2^+ = Q_2' + Up'Q_1' + UpQ_2Q_1 + UpQ_3Q_1$

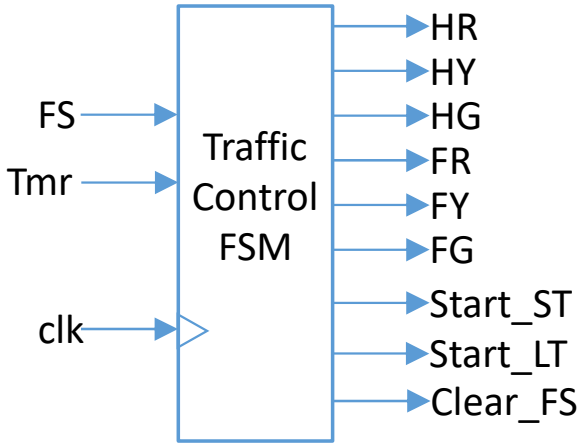
$Q_1^+ = Up'Q_3 + Q_2Q_1' + UpQ_2$

1. The reset and preset values are asserted at the start to bring the counter to state 0011
2. Once the normal operation starts, they are de-asserted, i.e., they are set to 1
3. Note that both reset_n and preset_n are active low.
   a. This means that when preset_n = 0, it will preset, i.e, set the output of the Flop to 1
   b. Likewise, when reset_n = 0, it will reset, i.e., set the output of the Flop to 0
   c. Finally, when both reset_n and preset_n are made 1, they are de-asserted, i.e., they do not affect the output

© Ahmed Hemani, KTH, Stockholm

Q13. 5p



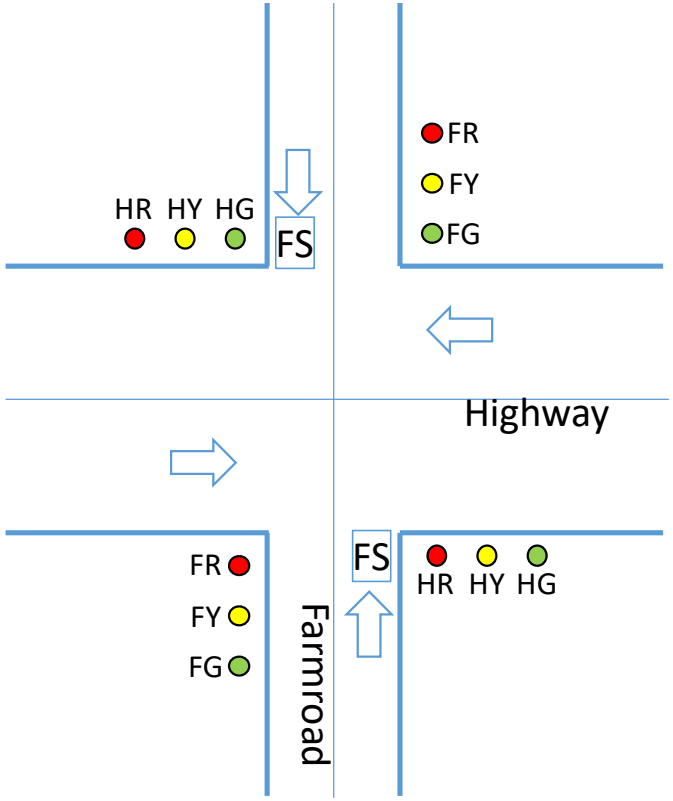


HR:	Output – Highway Red Signal. Highway Traffic Blocked
HY:	Output – Highway Yello Signal. Highway Traffic Prepare to Stop
HG:	Output – Highway Green Signal. Highway Traffic Allowed
FR:	Output – Farmroad Red Signal. Farm road Traffic Blocked
FY:	Output – Farmroad Yello Signal. Farm road Traffic Prepare to Stop
FG:	Output – Farmroad Green Signal. Farm road Traffic Allowed
Start_ST:	Output – Start Short Timer. A pulse that lasts for one clock cycle
Start_LT:	Output – Start Long Timer. A pulse that lasts for one clock cycle
Tmr:	Input – When Tmr = 0, Short or Long Timer time has elapsed
FS:	Input – Farm road Sensor. When FS = 1, there is traffic on the Farm road
Clear_FS:	Output – Reset Farm Sensor
clk:	Input – All transitions in Traffic Control FSM are synchronous to clk

© Ahmed Hemani, KTH, Stockholm

Question:

A finite state machine (FSM) traffic light controller controls the intersection of a Highway with that of a small Farm road. Both roads are controlled by Red, Yellow and Green Lights. The illustration of the two roads, their signals and their operation is shown in the diagram and text below.



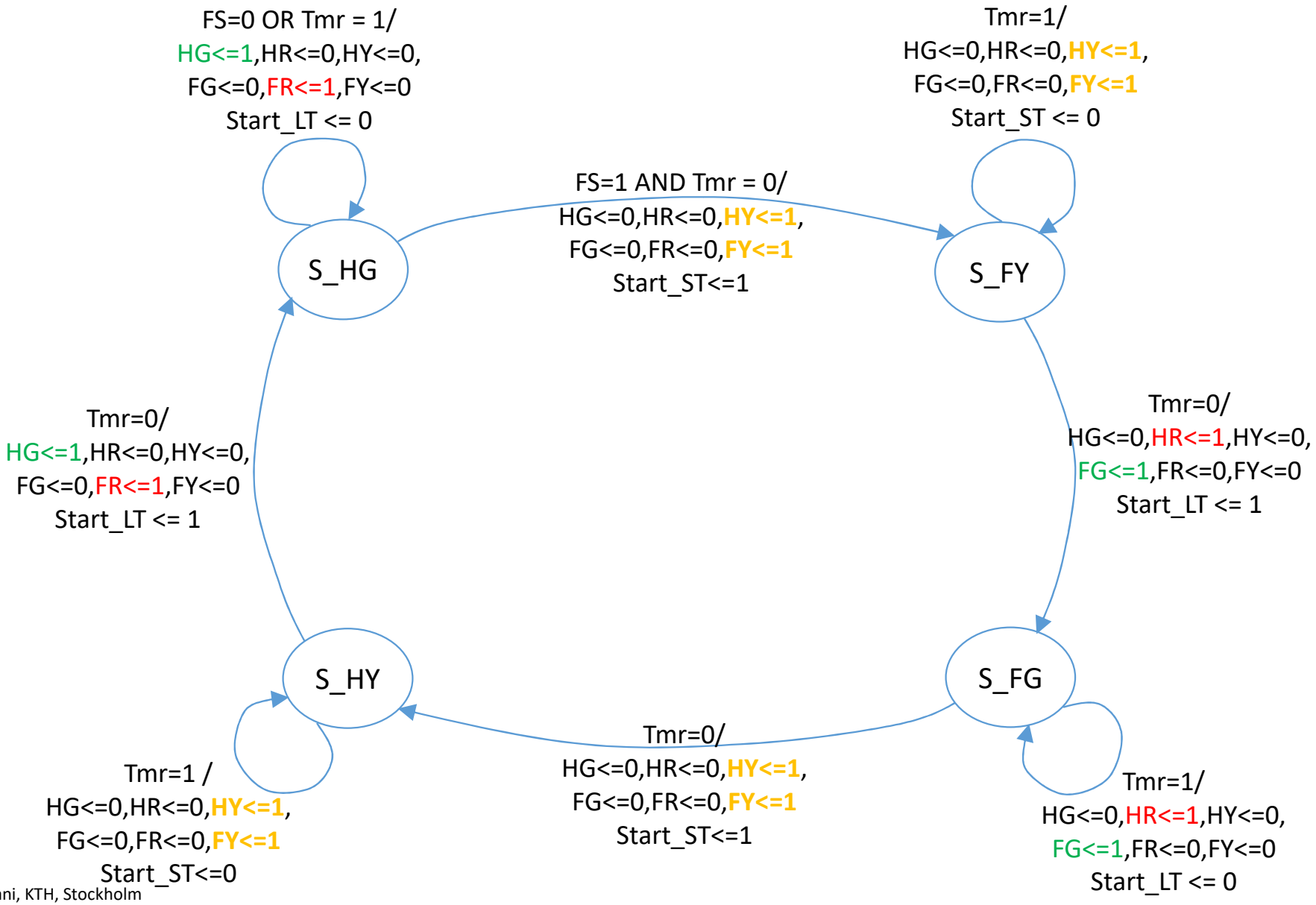
Design the traffic control FSM, taking the following into account:

- A traffic signal consists of three lights –Red, Yellow and Green. For each road and traffic direction, there is a traffic signal as shown in the diagram above.
- The behaviour of the two traffic signals for the two directions on the Highway is identical, which is also true for the two traffic signals for the two directions on the Farm road.
- For each traffic signal, only one light can be ON; the other two are OFF
- To give priority to the Highway, by default, the Highway green light HG is ON, and the Farm road red light FR is ON.

1. When the Highway traffic is enabled, if FS becomes 1, the traffic control FSM turns OFF HG and FR, turns ON HY and FY, and starts a short timer by setting Start_ST to 1.
2. When the short duration has elapsed, indicated by Tmr = 0, the traffic control FSM turns OFF the HY and FY lights, turns ON the HR and FG lights and at the same time starts the long timer by setting Start_LT to 1.
3. When the long duration has elapsed, indicated again by Tmr = 0, it is time to block the Farm road traffic again and enable the Highway traffic. This is done by turning OFF HR and FG and turning ON HY and FY, and starting the short timer by setting Start_ST to 1
4. When the short duration has elapsed, indicated by Tmr=0, the traffic control FSM turns OFF the HY and FY lights, turns ON the HG and FR lights and starts a long-term timer with a cycle-long clock pulse.
5. When Tmr=0, indicating that the long duration has elapsed and it is time to wait for the FS to become 1 again. And *when* it does, repeat the steps from 1 above.

Draw the state diagram and the state-transition table for a Mealy FSM

© Ahmed Hemani, KTH, Stockholm



© Ahmed Hemani, KTH, Stockholm

Inputs		Present State	Next State	Outputs							
FS	**Tmr**	**Q1 Q0**	**Q1+ Q0+**	**Start_ST**	**Start_LT**	**HG**	**HY**	**HR**	**FG**	**FY**	**FR**
-	1	S_HG	S_HG	0	0	1	0	0	0	0	1
0	-	S_HG	S_HG	0	0	1	0	0	0	0	1
1	0	S_HG	S_FY	1	0	0	1	0	0	1	0
-	1	S_FY	S_FY	0	0	0	1	0	0	1	0
-	0	S_FY	S_FG	0	1	0	0	1	1	0	0
-	1	S_FG	S_FG	0	0	0	0	1	1	0	0
-	0	S_FG	S_HY	1	0	0	1	0	0	1	0
-	1	S_HY	S_HY	0	0	0	1	0	0	1	0
-	0	S_HY	S_HG	0	1	1	0	0	0	0	1

© Ahmed Hemani, KTH, Stockholm